SIMATS ENGINEERING
ASSESSMENT TOOLS

COURSE NAME: Theory of Computation COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO3 Construct regular expressions, and use the pumping lemma and closure properties to analyze regular languages. (BL – 3)

Assessment Tool Weightage: 20%

QUESTIONS: 5
Duration: 1 Hour

Total Marks:25

CASE STUDY

Code of Conduct:

I A Manoj Reddy 192311171 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Manoj

1 Consider a case study in parsing nested structured expressions using a Context-Free Grammar (CFG). Given the grammar

$S \rightarrow (L) \mid a$

$L \rightarrow L, S \mid S,$

SIMATS ENGINEERING
ASSESSMENT TOOLS

1. **Designing a Grammar for Arithmetic Expressions**
   A compiler development team is creating a parser for a calculator application that supports addition, subtraction, multiplication, division, and parentheses. The initial grammar produces ambiguity for expressions such as a+b*c.

Case Study Question:
Develop an optimized context-free grammar (CFG) that correctly enforces operator precedence and associativity for the given arithmetic expressions. Explain how your grammar removes ambiguity and supports efficient parsing.

---

2. **Mini Programming Language Parser Development**
   A startup is designing a small scripting language for automating IoT devices. The language supports variable declarations, conditional statements, and loops. However, the parser frequently reports syntax conflicts during compilation.

Case Study Question:
Construct a CFG for the scripting language constructs below and optimize it to eliminate left recursion and reduce parsing conflicts. Demonstrate how the optimized grammar improves parser efficiency.

---

3. **Grammar Optimization for SQL Query Validation**
   A database company is developing a query validation tool for simplified SQL commands such as SELECT, FROM, and WHERE. The original grammar contains redundant productions that increase parsing time.

Case Study Question:
Develop a CFG for simplified SQL query syntax and optimize the grammar by removing unnecessary productions and ambiguity. Discuss how the optimized grammar helps in faster syntax analysis.

---

4. **Natural Language Processing Chatbot**
   A chatbot system processes simple English sentences such as:
   Subject + Verb + Object
   Example: "Students learn TOC."
   The initial CFG incorrectly accepts grammatically invalid sentences.

Case Study Question:
Design a CFG for the chatbot sentence structure and optimize it to improve language validation accuracy. Explain how the grammar ensures correct sentence formation and rejects invalid inputs.

---

5. **Parser Design for HTML-like Markup Language**
   A web editor company is building a parser for a simplified markup language with opening and closing tags. The existing grammar fails to validate nested tags correctly.

Case Study Question:
Develop and optimize a CFG for validating properly nested tags in the markup language. Show how the grammar supports recursive nesting and improves parsing reliability.

SIMATS ENGINEERING
ASSESSMENT TOOLS

1. Designing a Grammar for Arithmetic Expressions

Case Study

A compiler development team is creating a parser for a calculator application that supports addition, subtraction, multiplication, division, and parentheses. The initial grammar produces ambiguity for expressions such as a+b*c.

Answer

Ambiguous Grammar

$$E \rightarrow E + E \mid E * E \mid (E) \mid id$$

This grammar is ambiguous because the expression a+b*c can have multiple parse trees.

Optimized CFG

$$E \rightarrow E + T \mid E - T \mid T$$
$$T \rightarrow T * F \mid T/F \mid F$$
$$F \rightarrow (E) \mid id$$

Optimization Performed

- Separate non-terminals for precedence levels.
- T handles multiplication/division.
- E handles addition/subtraction.
- Removes ambiguity.
- Supports efficient top-down or bottom-up parsing.

---

2. Mini Programming Language Parser Development

Case Study

A startup is designing a small scripting language for automating IoT devices. The parser frequently reports syntax conflicts during compilation.

Answer

Initial Grammar with Left Recursion

$$Stmt \rightarrow Stmt\ stmt \mid stmt$$

Optimized CFG

$$Stmt \rightarrow stmt\ Stmt'$$
$$Stmt' \rightarrow stmt\ Stmt' \mid \epsilon$$

CFG for Language Constructs

$$Loop \rightarrow while(condition)\{Stmt\}$$
$$Cond \rightarrow if(condition)\{Stmt\}$$

Optimization Performed

- Eliminated left recursion.
- Reduced shift-reduce conflicts.
- Improved predictive parsing.

---

3. Grammar Optimization for SQL Query Validation

Case Study

A database company is developing a query validation tool for simplified SQL commands.

Answer

Initial Grammar

$$Q \rightarrow SELECT\ A\ FROM\ T\ WHERE\ C$$

Optimized CFG

$$Q \rightarrow SELECT\ Fields\ FROM\ Table\ OptWhere$$
$$OptWhere \rightarrow WHERE\ Condition \mid \epsilon$$

Optimization Performed

- Introduced optional productions.
- Reduced redundancy.
- Simplified parser design.
- Improved syntax validation speed.

Example Accepted Query

SELECT name FROM student WHERE mark > 50

---

4. Natural Language Processing Chatbot

Case Study

A chatbot processes simple English sentences like "Students learn TOC."

Answer

CFG Design

$$S \rightarrow NP\ VP$$
$$NP \rightarrow Noun$$
$$VP \rightarrow Verb\ NP$$
$$Noun \rightarrow Students \mid TOC$$
$$Verb \rightarrow learn$$

Optimization Performed

- Restricted sentence order.
- Invalid patterns are rejected.
- Improved grammatical correctness.

Valid Sentence

Students learn TOC

Invalid Sentence

Learn Students TOC

---

5. Parser Design for HTML-like Markup Language

Case Study

A web editor company is building a parser for a simplified markup language with nested tags.

Answer

CFG for Nested Tags

$$S \rightarrow < tag > S < /tag > \mid text \mid \epsilon$$

Example

```
<tag>
  <tag>text</tag>
</tag>
```

Optimization Performed

- Recursive production supports nesting.
- Ensures proper opening and closing tags.
- Detects invalid nesting structures.

Invalid Example

<tag>text</tag>

Benefit

SIMATS ENGINEERING
ASSESSMENT TOOLS

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

88/100

Designing a Grammar for Arithmetic Expression:-

Ambigous Grammar:-

$$E \rightarrow E+E \mid E-E \mid E*E \mid E/E \mid (E) \mid id$$

→ This grammar is ambigous because the expression a+b*c can have multiple parse trees.

Optimized CFG:-

$$E \rightarrow E+T \mid E-T \mid T$$

$$T \rightarrow T*F \mid T/F \mid F$$

$$T \rightarrow (E) \mid id$$

Example:- a+b*c

is represented as a+(b*c)

2) Mini Programming language parser development

Case Study:-

A startup is developing a scripting language IoT devices. Supporting variable declartion, Conduct statement and loops.

initial grammar:-

Stmt → Simple Stmt Stmt'

Stmt' → simple Stmt Stmt' | ∈

Conditional statement:

if Stmt → if (condition) { Stmt }

loop statement:-

loop → while (condition) { Stmt }

variable declartion:-

Dec → type id;

Example:- int x;

if (Condition) { x;

while (condition) {

x';

} for } SQL query:-

Validation:-

A database Company is developing a query validation tool for simplified SQL query validation.

intial grammar:-

Q → Select A from T where C

Optimized CFG:-

Q → Select fields FROM Table opt

Grammar optimization

Opt → WHERE Condition | ∈

Fields → id | id , Fields

Table → id

Example :- accepted queries

SELECT name FROM Student

SELECT name FROM Student, where marks.

4) NLP processing chatbot:

A chatbot Proces simple English Sentence

CFG:- S → NPVP | Noun - Student

NP → Naun | verb - learn

VP → verb NP

valid sentence

Students learn TOC

derivation: S ⇒ NPVP

Students verb NP

Students learn TOC

5) Parser Design FOV HTML:-

A web editor Company is building a Parser for simplified.

S → <tag> S </tag> | text | ∈

<tag>

<tag> <text> </tag>

</tag>

optimized performed:-

- Recursive production supports nested tags
- Ensure matching opening and closing tags
- Detect incorrect nesting
- Improves parsing reactivity.

A possible derivation is:

S

=> Element

=> <tag> Content </tag>

=> <tag> Element Content </tag>

=> <tag> Content Content </tag>

=> <tag> text </tag>

The recursive rule:-

Content → Element Content